Optimized Model Selection for Embedded Drowsiness Detection: A Comparative Study of MobileNetV2 and EfficientNet-B0 on ESP32-CAM

Salwa Aulia Farida Yudistira
School of Electrical Engineering
Telkom University
Bandung, Indonesia
salwaafariday@gmail.com

Ardhian Wisnu Kartika
School of Electrical Engineering
Telkom University
Bandung, Indonesia
wisnuardhian969@gmail.com

Chintia Febriani
School of Electrical Engineering
Telkom University
Bandung, Indonesia
chintiafebriani@student.telkomuniversity.ac.id

Nyoman Bogi Aditya Karna
The Univ CoE Intelligent Sensing-IoT
Telkom University
Bandung, Indonesia
aditya@telkomuniversity.ac.id

Inung Wijayanto
School of Electrical Engineering
Telkom University
Bandung, Indonesia
iwijayanto@telkomuniversity.ac.id

2025 1st International Conference on Artificial Intelligence Technology (ICoAIT) | 979-8-3315-8679-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICoAIT67446.2025.11309086

***Abstract*— Efficient driver drowsiness detection is a critical safety application, as drowsy driving is a leading cause of road accidents. Lightweight CNNs such as MobileNetV2 have been specifically designed for mobile and embedded vision tasks, offering high accuracy with low computational cost. In this study, we conduct a comparative analysis of three INT8-quantized CNN models for drowsiness detection on a resource-constrained device (ESP32-CAM): MobileNetV2 multi-output, MobileNetV2 single-output, and EfficientNet-B0. All models were trained on the same drowsiness dataset (collected via Roboflow) and evaluated using training and validation accuracy and loss, as well as confusion-matrix metrics (precision, recall, F1-score). Our results show that the MobileNetV2 single-output model achieved 96% accuracy in FLOAT32 and 93% in INT8, offering a strong balance between performance and efficiency, making it suitable for ESP32-CAM deployment. In contrast, the multi-output MobileNetV2 reached 95% accuracy in FLOAT32 but dropped to 30% in INT8, rendering it unsuitable for constrained devices. EfficientNet-B0 outperformed both in FLOAT32 with 98% accuracy and maintained 96% in INT8, but its larger model size and memory demands exceeded ESP32-CAM limitations. The main contributions of this work are: (1) a comparative analysis of three lightweight CNNs for embedded drowsiness detection, (2) evaluation of INT8 quantization on ESP32-CAM, (3) identification of MobileNetV2 single-output as the most deployable model, and (4) practical design insights for TinyML-based driver monitoring.**

Keywords— Driver Drowsiness Detection, TinyML, MobileNetV2, EfficientNet-B0, Model Quantization, ESP32-CAM, Edge AI

I. INTRODUCTION (*HEADING 1*)

Driver drowsiness detection is essential for road safety, as it is a leading cause of traffic accidents[1]. Conventional methods based on eye-blink or PERCLOS heuristics have gradually been overtaken by deep learning approaches, especially CNNs, which achieve 92-97% accuracy on benchmark drowsiness datasets [1]. However, these models are typically too large to run on microcontroller-based platforms. It is well known that devices like the ESP32-CAM have very limited memory often <1 MiB RAM and require aggressive model optimization to perform deep learning inference [2]. TinyML approaches address this by using lightweight networks and quantization. For example, a quantized MobileNetV2 can achieve 99.64% accuracy with a model size of only 1.55 MB [3]. Similarly, an edge-based TinyML system for drowsiness detection achieved 98% accuracy using quantized models, as reported in [4]. On the other hand, hybrid models such as the EffRes-DrowsyNet (combining EfficientNet-B0 and ResNet50) reach very high accuracy (97-98%) [5], but their complexity is too high for real-time embedded use.

One of the main challenges in deploying driver drowsiness detection systems on low-power embedded devices is selecting a convolutional neural network (CNN) model that offers both high accuracy and a compact size suitable for resource-constrained hardware [6]. To address this problem, we compare three CNN architectures: MobileNetV2 with multi-output, standard MobileNetV2, and EfficientNet-B0. All models are quantized to 8-bit integer format (INT8) using TensorFlow Lite in order to reduce model size and improve inference efficiency. These models are trained on a common drowsiness image dataset and evaluated based on their training and validation accuracy, loss curves, and confusion matrix metrics (including precision, recall, and F1-score). In addition to performance evaluation, we also measure the model sizes after quantization and test their deployability on an ESP32-CAM device to assess real-world feasibility. The main contributions of this study are as follows:

1. Presenting a comparative performance analysis of three lightweight CNN models for drowsiness detection in embedded environments.
2. Implementing full INT8 quantization using TensorFlow Lite and evaluating the deployment feasibility of each model.

979-8-3315-8679-9/25/$31.00©2025 IEEE

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:38:40 UTC from IEEE Xplore. Restrictions apply.

3. Identifying the standard MobileNetV2 model as the best trade-off between accuracy and model size, suitable for on-device deployment.
4. Providing practical design guidelines and benchmark results to support the development of TinyML-based driver monitoring systems on resource-constrained hardware.

II. Related Works

Several recent studies have leveraged lightweight CNN architectures for real-time drowsiness detection, especially on low-power embedded devices. A prominent approach is the use of TinyML, which enables machine learning on constrained hardware through model compression, quantization, and pruning [6]. EffRes-DrowsyNet, a hybrid CNN combining EfficientNet-B0 and ResNet50, achieved 97.7% accuracy, 98.1% precision, and 97.3% recall on the SUST-DDD dataset [5]. These results confirm that EfficientNet-B0 offers high representational power for drowsiness cues. Similarly, a recent review highlighted that modern transformers such as ViT and Swin can exceed 99% accuracy, while CNNs like VGG16 and ResNet50V2 still achieve 92–97% [1].

A real-time drowsiness detection system based on TinyML was developed using the FOMO (Fast Object Detection for Mobile Only) algorithm, which is specifically designed for edge devices with limited computational resources [4]. The system detects key drowsiness indicators, including eye state (open or closed) and yawning, by training a lightweight object detection model on annotated facial datasets. To enable deployment on constrained devices such as the Raspberry Pi Pico and ESP32-S3, the model was optimized using TensorFlow Lite (TFLite) and fully quantized to the INT8 format. Evaluation results showed a validation accuracy of 98.2% in float32 and 97.9% after INT8 quantization, with a significantly reduced model size and fast inference time suitable for real-time operation [4].

Traditional CNNs such as VGG16, ResNet50, and MobileNet have been compared with Vision Transformers (ViT) for drowsiness detection [1]. ViTs achieved over 99% accuracy, outperforming CNNs in most metrics. However, ViTs require high memory and computational resources, making them impractical for low-power devices like ESP32-CAM. In contrast, CNNs like MobileNet achieved 92–97% accuracy, while remaining lightweight, fast, and deployable on microcontrollers after quantization. The study highlights a trade-off: ViTs offer higher accuracy, but CNNs are more suitable for real-time embedded applications [1].

Critical to all embedded approaches is model compression. Works on TinyML emphasize quantization as essential for reducing memory and computation [2], [3]. EfficientNet itself was designed for compound scaling of MobileNet-like blocks to achieve state-of-the-art accuracy with fewer parameters. In summary, lightweight CNNs (MobileNetV2, EfficientNet) are effective for drowsiness detection high accuracy [1], [5], and TinyML strategies (quantization, small models) make them feasible for resource-limited devices [2], [3].

However, most prior works did not account for the hardware limitations of real-world embedded platforms such as ESP32-CAM. According to a study, many microcontrollers offer less than 1 MiB of SRAM, which severely limits the deployable model size [7]. To overcome this, recent studies apply full INT8 quantization, converting model parameters to uint8 format to drastically reduce memory usage up to 4 times smaller than float32 models while maintaining performance [8]. This study contributes by comparing three architectures:

1. MobileNetV2 single-output (binary eye state classification);
2. MobileNetV2 multi-output (multi-task);
3. EfficientNetB0, a compound-scaled model.

The evaluation not only includes accuracy, precision, recall, and F1-score, but also post-quantization model size and feasibility for deployment on ESP32-CAM. Final results show that only the MobileNetV2 single-output model 1.1 MB could be successfully deployed and run on the ESP32-CAM, while other models exceeded memory limits despite having higher accuracy.

III. System Models

The system developed in this study is designed to detect signs of drowsiness based on eye status (open or close), while considering hardware limitations such as those of the ESP32-CAM, and comprises four main components: (i) Dataset, (ii) Model Architecture, (iii) Prediction Output on PC, and (iv) Optimization to Implementation on Device.

A. Datasets

A driver facial image dataset annotated for eye status classification (drowsy or awake) was used in this study. The dataset was processed using the Roboflow platform, which supports labeling, preprocessing, and export in various formats, including TensorFlow Object Detection [9]. The Roboflow platform also enabled the application of data preprocessing and augmentation techniques, including cropping with a maximum zoom of 20%, rotation between -15° and +15°, and the addition of noise up to 0.7% of the image [10]. The result of the preprocessing and augmentation pipeline applied through the Roboflow platform is shown in Fig. 1. The dataset reflects real-world driver conditions, both alert and drowsy. It was divided into 70% training, 20% validation, and 10% testing sets, following standard practices in deep learning model training [9]. This partitioning approach helps reduce training bias, as demonstrated in [9]. All images were converted to greyscale to reduce input complexity and data size, and resized to 96×96 pixels an input dimension commonly used in lightweight CNN implementations compatible with MobileNetV2 or EfficientNet architectures [11], [12], [13].



Fig. 1. Augmented Images from Roboflow

B. Model Architecture

This study compares three lightweight deep learning architectures based on Convolutional Neural Network (CNN) that are known for their efficiency and widespread use in image recognition systems on edge devices: MobileNetV2 multi-output, MobileNetV2 single-output, and EfficientNet-

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:38:40 UTC from IEEE Xplore. Restrictions apply.

B0. Convolutional Neural Network (CNN) is one of the models commonly used in deep learning. The structure of a CNN consists of two main components: the feature learning layers and the fully connected layers. The feature learning component includes several layers, such as convolutional layers, activation functions (typically ReLU), average pooling layers, and dropout regularization [6]. CNNs operate in a hierarchical manner, where the output from an earlier convolutional layer serves as the input for the subsequent convolutional layer [14]. All architectures were implemented using TensorFlow/Keras and subsequently converted to TensorFlow Lite format with INT8 quantization for evaluation and deployment on resource-constrained devices such as the ESP32-CAM [8].

C. MobileNetV2 Multi-Output

MobileNetV2 is a CNN architecture designed for high efficiency in terms of inference speed and parameter count. It utilizes Inverted Residual blocks and Linear Bottlenecks an architectural strategy that employs depthwise separable convolutions in the middle layers and eliminates non-linear activations (ReLU6) after bottleneck layers [12]. This design has been shown to preserve crucial spatial information while significantly reducing model size. The multi-output architecture extends the single-output classification by adding a regression head for predicting eye region coordinates (bounding box), enabling spatial localization and state classification simultaneously, as illustrated in Fig. 2.

```
# Branch 1: Bounding Box Regression head
bbox_dense1 = layers.Dense(64,
activation='relu')(x)
bbox_output = layers.Dense(4,
activation='linear',
name='bbox_output')(bbox_dense1)

# Branch 2: Classification head
class_dense1 = layers.Dense(64,
activation='relu')(x)
class_output = layers.Dense(1,
activation='sigmoid',
name='class_output')(class_dense1)

# Define multi-output model
model = Model(inputs=inputs,
outputs=[bbox_output, class_output])
```

Fig. 2. Multi-Output Model for Eye Detection and Classification

A related study explored facial expression recognition using an enhanced MobileNetV2 architecture with integrated attention mechanisms and residual structures. The model implicitly supports multi-output functionality by combining facial feature detection and classification tasks, demonstrating the flexibility of MobileNetV2 in multi-task facial analysis applications [15].

Recent work has demonstrated that multi-output CNN architectures are effective in detecting complex driver states such as fatigue, stress, and drowsiness, especially in in-vehicle camera systems. However, in memory-constrained environments, the added computational and memory overhead introduced by multiple output heads may limit deployment feasibility [16].

D. MobilenetV2 Single-Output

This version adopts the basic MobileNetV2 architecture with a single output head for binary classification, predicting only the eye state: open or closed. This approach is computationally lightweight and well-suited for deployment on edge devices with limited RAM and storage capacity, such as the ESP32-CAM. MobileNetV2 has proven highly effective for lightweight image classification tasks and has been widely adopted in real-time systems on IoT devices. For example, MobileNetV2 has been evaluated for real-time face classification on mobile and embedded platforms [17].

In our study, the architecture processes grayscale input images with a resolution of 96×96 pixels, selected to balance feature quality with a minimal model footprint. The final layer is a Dense layer with a softmax activation function, producing class probabilities for the two eye states [18]. After applying full integer quantization, the model size is approximately 1.2–1.8 MB. With careful memory configuration, it can be deployed on the ESP32-CAM. Its main advantages are fast inference time and a minimal memory footprint, making it an ideal choice for edge inferencing. Fig. 3. Construction of the MobileNetV2 single-output model configured with a 96×96 grayscale input, width multiplier (α = 0.05), and global average pooling for feature extraction.

```
base_model = MobileNetV2(input_shape=(96, 96, 1), alpha=0.05,
include_top=False,
weights="mobilenetV2_0.05_96x96_greyscale_weights.h5",
input_tensor=Input(shape=(96, 96, 1)), pooling="avg")
```

Fig. 3. MobileNetV2 Single-Output Model Configuration

E. EfficientNet-B0

EfficientNet-B0 is the smallest variant in the EfficientNet family, developed using a compound scaling strategy that scales depth, width, and input resolution simultaneously for optimal model efficiency [13]. Its architecture is built upon MBConv (Mobile Inverted Bottleneck) blocks combined with a squeeze-and-excitation mechanism, enabling a balance between high classification accuracy and compact parameter size [19].

This model has demonstrated strong performance across various classification benchmarks, with excellent parameter efficiency [20]. However, despite its efficiency-oriented design, the model size after full INT8 quantization remains approximately 5–6 MB, which exceeds the available memory of resource-constrained devices such as the ESP32-CAM. This limitation underscores the need for careful consideration of the trade-off between model accuracy and footprint when deploying deep learning models on low-power microcontroller platforms [8]. The model construction of EfficientNet-B0, utilizing a 96×96 RGB input and ImageNet pre-trained weights, is presented in Fig. 4.

```
base_model = tf.keras.applications.EfficientNetB0(include_top=False,
weights='imagenet', input_shape=(96, 96, 3))
```

Fig. 4. EfficientNet-B0 Model Configuration

F. Quantization

To enable deployment on memory-constrained microcontroller platforms, all models in this study were converted to TensorFlow Lite format and optimized using 8-bit post-training quantization (INT8). Quantization is a crucial step in TinyML workflows, as it significantly reduces model size and inference latency, while lowering power consumption key requirements for embedded systems such as the ESP32-CAM [8], [21].

Post-training quantization converts 32-bit floating point weights and activations into 8-bit integers, enabling up to 4× compression without retraining. This process was conducted

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:38:40 UTC from IEEE Xplore. Restrictions apply.

using TensorFlow's tf.lite.TFLiteConverter with full-integer quantization. Previous studies confirm that INT8 quantization can maintain comparable accuracy to the original model while significantly reducing storage and computational demands [22], [23], [24]. After quantization, models were evaluated on a desktop using validation accuracy, loss, and confusion matrix (precision, recall, F1-score) to ensure acceptable performance degradation before deployment.

IV. Performance Evaluation

This section presents the evaluation results of three models MobileNetV2 Multi-Output, MobileNetV2 Single-Output, and EfficientNet-B0 for drowsiness detection. Each model was tested in FLOAT32 and INT8 TFLite formats using standard metrics such as accuracy, precision, recall, and F1-score. The analysis highlights performance trade-offs and hardware suitability, particularly for deployment on resource-constrained devices like the ESP32-CAM.

A. MobilenetV2 Multi-output

The MobileNetV2 multi-output model, designed for simultaneous eye position detection and eye state classification, was evaluated in both FLOAT32 and INT8 formats. The FLOAT32 model achieved a satisfactory accuracy of 95%, with strong class-wise performance precision and recall of 0.97 and 0.93 for the drowsy class, and 0.94 and 0.97 for the awake class, respectively. As illustrated in Fig. 5, the confusion matrix shows a limited number of misclassifications (32 drowsy and 14 awake), indicating the model's effective learning of both spatial and categorical features. However, despite its predictive strength, the FLOAT32 model remains unsuitable for deployment on resource-constrained devices such as the ESP32-CAM, which provides only 512 KB of SRAM and 4 MB of flash memory, and lacks hardware support for floating-point operations. The model's size exceeding 14 MB renders it incompatible with such platforms.

In contrast, the INT8 quantized version experienced a substantial drop in performance, with an overall accuracy of only 30% and a combined F1-score of 0.46. Notably, the awake class was entirely misclassified, as indicated by a recall and precision of 0.00. As shown in Fig. 5, the confusion matrix reveals that nearly all predictions were biased toward the drowsy class. This severe degradation is primarily attributed to two engineering limitations. First, INT8 quantization significantly reduces numerical precision, impairing the model's ability to capture fine-grained spatial features required for accurate eye localization. Second, the added complexity of multi-output learning results in a larger architecture that, even after quantization, remains above 6 MB in size still exceeding the ESP32-CAM's memory constraints. Therefore, while quantization typically improves deployability on embedded systems, in this case it severely compromises model accuracy and remains infeasible for practical implementation.

B. MobileNetV2 Single-output

The MobileNetV2 model was trained over 64 epochs using 21,294 balanced images (10,249 drowsy and 11,045 awake) and evaluated on a test set of 1,015 samples, with results shown in Fig. 6. The FLOAT32 version achieved 96% accuracy with strong metrics for both classes: drowsy (precision 0.97, recall 0.95, F1-score 0.96) and awake (precision 0.96, recall 0.97, F1-score 0.96). Only 22 drowsy and 16 awake samples were misclassified. The model's 32-bit precision enables detailed feature extraction but results in a large model size and high compute demand, making it unsuitable for microcontrollers like ESP32-CAM.

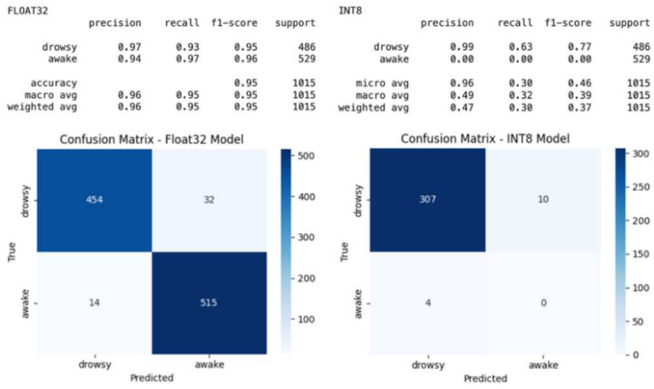


Fig. 5. MobileNetV2 multi-output performance

The INT8 quantized version reached 93% accuracy slightly lower but still robust. It achieved 0.90 precision and 0.96 recall for drowsy, and 0.96 precision and 0.90 recall for awake, with an F1-score of 0.93 for both. Although it misclassified 54 awake samples, the performance remains consistent. Quantization reduces weights from 32-bit to 8-bit, shrinking the model and speeding up inference with lower power use. Given the ESP32-CAM's limited memory (512 KB SRAM, 4 MB flash) and lack of floating-point support, the INT8 model is more suitable, offering an effective balance between size, speed, and accuracy for real-time embedded deployment.

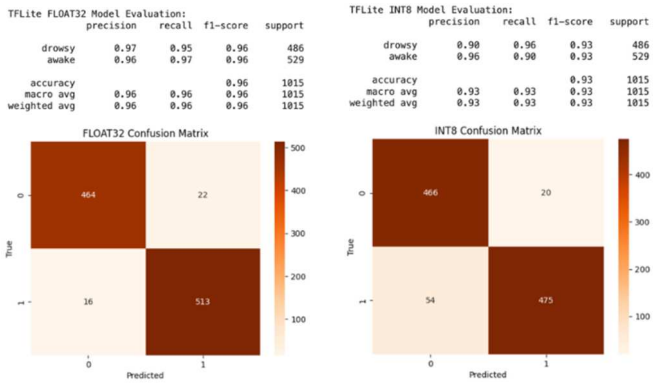


Fig. 6. MobilenetV2 single-output performance

C. EfficientNet-B0

The EfficientNet-B0 FLOAT32 model demonstrates excellent classification performance for detecting drowsy and awake states, achieving an accuracy of 98% as shown in Fig. 7 (left). Both classes exhibit high precision and recall (drowsy: 0.99 and 0.98; awake: 0.98 and 0.99), with only 10 drowsy and 6 awake samples misclassified, indicating a well-generalized model. This performance is enabled by the 32-bit floating-point computation, which effectively captures complex facial features. However, the FLOAT32 model has a substantial size of approximately 20MB and high inference demands, making it unsuitable for deployment on microcontroller-class devices like the ESP32-CAM, which is limited to 512KB SRAM and 4MB flash, and lacks floating-point hardware support.

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:38:40 UTC from IEEE Xplore. Restrictions apply.

The EfficientNet-B0 INT8 quantized model, shown in Fig. 7 (right), retains strong performance with 96% accuracy. Despite a slight drop, it still maintains good precision and recall (drowsy: 0.96 and 0.95; awake: 0.95 and 0.97), with 26 drowsy and 18 awake samples misclassified. Quantization to 8-bit integers significantly reduces computational overhead and improves inference speed, which is ideal for edge devices. However, the model's size after quantization remains relatively large around 10–12MB, exceeding the available flash storage on the ESP32-CAM. Additionally, EfficientNet's deeper architecture requires more RAM during inference. Thus, despite strong accuracy metrics, the INT8 version of EfficientNet-B0 is still impractical for ESP32-CAM deployment due to memory and hardware limitations.

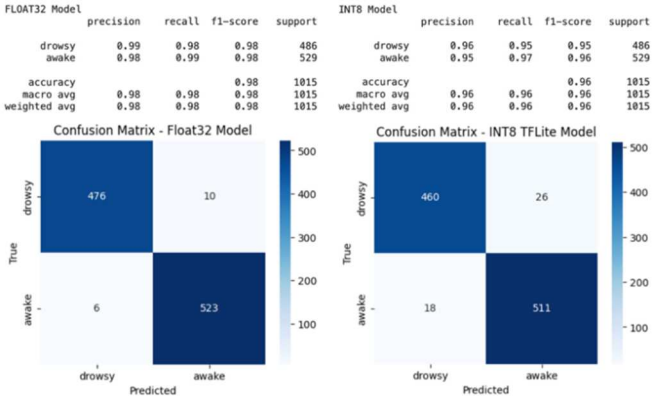


Fig. 7. EfficientNet-B0 performance

D. Model Deployability & Resource Constraints

Three candidate models, namely MobileNetV2 Single-Output, MobileNetV2 Multi-Output, and EfficientNet-B0, were converted and deployed on the ESP32-CAM platform using TensorFlow Lite Micro. During conversion, TFLite file sizes and peak RAM requirements (model activation and input buffer) were measured, followed by flashing attempts on the device (4 MB flash, ~520 KB SRAM). Only the MobileNetV2 Single-Output model, after post-training INT8 quantization, successfully produced a binary that fit within both flash and run-time memory constraints. In contrast, MobileNetV2 Multi-Output and EfficientNet-B0 exceeded the available flash or SRAM even after quantization, failing either at the linking/flashing stage or during runtime allocation. These results highlight a critical practical limitation in model selection for highly resource-constrained microcontrollers and justify the focus on MobileNetV2 Single-Output for real-time on-device inference, as shown in Table I.

TABLE I. COMPARISON OF MODEL PERFORMANCE

Model	Params (M)	.tflite size (FP32)	.tflite size (INT8)	.h (MB)	Status on ESP32-CAM
MobileNetV2 Single-Output	0.093	0.361 MB	0.198 MB	2 MB	Successful
MobileNetV2 Multi-Output	2.59	9.7 MB	2.9 MB	16 MB	Failed
EfficientNet-B0	4.05	15.3 MB	4.7 MB	25 MB	Failed

V. CONCLUSION

This study compared three Machine Learning architectures MobileNetV2 single-output, MobileNetV2 multi-output, and EfficientNet-B0 for deployment on the ESP32-CAM, a highly resource-constrained microcontroller (512 KB SRAM, 4 MB flash, no FPU). While all models achieved strong performance in their FLOAT32 formats (95-98% accuracy), only the MobileNetV2 single-output remained viable after INT8 quantization. It preserved high accuracy (93%) with a compact 0.198 MB TFLite size (2 MB .h), making it suitable for on-device storage and real-time inference. In contrast, the MobileNetV2 multi-output variant, despite achieving 95% accuracy in FLOAT32, dropped to ~30% post-quantization and exceeded 16 MB, rendering it infeasible. EfficientNet-B0 also maintained strong accuracy (98% FLOAT32, 96% INT8) but was too large (25 MB .h) for ESP32-CAM deployment.

These findings underscore that model selection for embedded systems must balance accuracy, architectural complexity, and quantization robustness. A model with high baseline performance can become unusable if it exceeds flash limits or suffers significant accuracy degradation after quantization. Future work should target lightweight, deployable architectures through techniques like structured pruning, knowledge distillation, or quantization-aware training. Hardware-aware neural architecture search and compilers such as TFLite Micro or TVM can further enhance deployability, enabling reliable, efficient ML on ultra-low-power edge devices.

REFERENCES

[1] O. F. Hassan, A. F. Ibrahim, A. Gomaa, M. A. Makhlouf, and B. Hafiz, "Real-time driver drowsiness detection using transformer architectures: a novel deep learning approach," *Sci Rep*, vol. 15, no. 1, Dec. 2025, doi: 10.1038/s41598-025-02111-x.

[2] P. E. Novac, G. B. Hacene, A. Pegatoquet, B. Miramond, and V. Gripon, "Quantization and deployment of deep neural networks on microcontrollers," *Sensors*, vol. 21, no. 9, May 2021, doi: 10.3390/s21092984.

[3] N. N. Alajlan and D. M. Ibrahim, "DDD TinyML: A TinyML-Based Driver Drowsiness Detection Model Using Deep Learning," *Sensors*, vol. 23, no. 12, Jun. 2023, doi: 10.3390/s23125696.

[4] S. Liawatimena and N. Isworo, "Driver Drowsiness Detection on Edge Device Using TinyML Framework," 2025. [Online]. Available: https://www.researchgate.net/publication/392359109

[5] S. H. Al-Gburi *et al.*, "EffRes-DrowsyNet: A Novel Hybrid Deep Learning Model Combining EfficientNetB0 and ResNet50 for Driver Drowsiness Detection," *Sensors*, vol. 25, no. 12, Jun. 2025, doi: 10.3390/s25123711.

[6] N. Karna, R. Kelikualiq, B. Aditya, M. A. Paramartha Putra, D. Rahyuni, and S. N. Hertiana, "Health Monitoring System in Smart Greenbox for Chili Plant Using Convolutional Neural Network," in *Proceedings of the 2024 IEEE International Conference on Internet of Things and Intelligence Systems (IoTaIS)*, Indonesia: IEEE, 2024, pp. 38–43.

[7] V. Rajapakse, I. Karunanayake, and N. Ahmed, "Intelligence at the Extreme Edge: A Survey on Reformable TinyML," Jan. 2023, doi: 10.1145/3583683.

[8] P. Warden and D. Situnayake, "TinyML Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers."

[9] N. Sujana, M. Malik Mutoffar, and A. Azzam Haryanto, "Performance Analysis of YOLOv8 Optimized with Roboflow for Emotional Facial Expression Detection Using Machine Learning" [in Indonesian], vol. 06, 2024.

[10] S. Mushtaq, K. Razzaq Malik, Z. Ahmad, and M. Sajid, "A Deep Learning Based Approach to Enhance Object Edge Detection for Office Surveillance System", doi: 10.56979/601/2023.

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:38:40 UTC from IEEE Xplore. Restrictions apply.

[11] E. P. Ndamung, A. A. Pekuwali, and R. T. Abineno, "Optimization of Rice Leaf Image Segmentation with Thresholding Method in Disease Identification" [in Indonesian], 2023.

[12] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," Mar. 2019, [Online]. Available: http://arxiv.org/abs/1801.04381

[13] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," Sep. 2020, [Online]. Available: http://arxiv.org/abs/1905.11946

[14] I. K. Trisiawan and Y. Yuliza, "Application of Multi-Label Image Classification Using Convolutional Neural Network (CNN) Method for Beverage Bottle Sorting" [in Indonesian], *Jurnal Teknologi Elektro*, vol. 13, no. 1, p. 48, Feb. 2022, doi: 10.22441/jte.2022.v13i1.009.

[15] Q. Zhu, H. Zhuang, M. Zhao, S. Xu, and R. Meng, "A study on expression recognition based on improved mobilenetV2 network," *Sci Rep*, vol. 14, no. 1, Dec. 2024, doi: 10.1038/s41598-024-58736-x.

[16] A. A. Almazroi, M. A. Alqarni, N. Aslam, and R. A. Shah, "Real-Time CNN-Based Driver Distraction & Drowsiness Detection System," *Intelligent Automation and Soft Computing*, vol. 37, no. 2, pp. 2153–2174, 2023, doi: 10.32604/iasc.2023.039732.

[17] P. Gupta, "Efficient Face Recognition via Transfer Learning with MobileNetV2 Architecture," *www.irjmets.com @International Research Journal of Modernization in Engineering*, 2755, [Online]. Available: www.irjmets.com

[18] F. Alzami, M. Naufal, H. Al Azies, S. Winarno, and M. A. Soeleman, "Time Distributed MobileNetV2 with Auto-CLAHE for Eye Region Drowsiness Detection in Low Light Conditions," 2024. [Online]. Available: www.ijacsa.thesai.org

[19] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-Excitation Networks," May 2019, [Online]. Available: http://arxiv.org/abs/1709.01507

[20] R. Nandaputra, "Implementation of Transfer Learning in Coconut Quality Classification Using Pre-trained EfficientNet Model" [in Indonesian], Undergraduate Thesis, Universitas Islam Negeri Syarif Hidayatullah, 2024.

[21] A. Musa, H. A. Kakudi, M. Hassan, M. Hamada, U. Umar, and M. L. Salisu, "Lightweight Deep Learning Models For Edge Devices-A Survey," 2025.

[22] E. Yvinec, "Efficient Neural Networks : Post Training Pruning and Quantization." [Online]. Available: https://theses.hal.science/tel-04496138v1

[23] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," Feb. 2016, [Online]. Available: http://arxiv.org/abs/1510.00149

[24] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," Dec. 2017, [Online]. Available: http://arxiv.org/abs/1712.05877

Authorized licensed use limited to: COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY. Downloaded on October 05,2026 at 06:38:40 UTC from IEEE Xplore. Restrictions apply.